

Current Recursion and Tensor-Network Execution in PYAMPLICOL

Purpose. This note summarizes the matrix-element generation strategy used by AMPLICOL and the corresponding PYAMPLICOL implementation. The common idea is to avoid explicit Feynman-diagram enumeration. Instead, matrix elements are assembled from a directed acyclic graph (DAG) of off-shell currents. AMPLICOL emits Fortran code that numerically sweeps this current table. PYAMPLICOL builds the same logical DAG in Python, lowers bounded pieces to tensor-network expressions handled by Spenso, compiles the resulting parametric scalar/vector expressions with Symbolica evaluators, and can now execute the staged sweep from Rust through the PyO3 runtime `rusticol`.

Current status. The generic LC/NLC/full-colour staged-DAG implementation is the production PYAMPLICOL workflow. Process generation writes schema-v2 artifacts loaded by `rusticol`; model parameters are runtime inputs and may be overridden from TOML. The default benchmark route is Rusticol with SymJIT O3 stage evaluators, stage-local inputs, output chunk size 128, batch size 64, ten Horner iterations, Symbolica’s default CPE choice, and enlarged Horner/common-pair limits. The result matrices are generated as separate $\text{\LaTeX}$ inputs, and the benchmark writers refresh `pyAmpliCol.pdf` after each completed cell.

1 The AmpliCol Current Recursion

For a fixed leading-color ordering, a current is labelled by the external subset it contains, the particle type carried by the off-shell line, and any spin/chirality information. Schematically, if I is a set or ordered interval of external colored legs, a current is computed as

$$\mathcal{J}_{t,\chi}^\alpha(I) = P^\alpha{}_\beta(t, p_I) \sum_{\substack{I=A \cup B \\ A \cap B = \emptyset}} \mathcal{V}_{\gamma\delta}^\beta(t_A, t_B \rightarrow t) \mathcal{J}_{t_A, \chi_A}^\gamma(A) \mathcal{J}_{t_B, \chi_B}^\delta(B). \quad (1)$$

Here t is the current type (gluon, quark, auxiliary tensor, etc.), χ is the Weyl chirality where relevant, $p_I = \sum_{i \in I} p_i$, P is the propagator, and $\mathcal{V}$ is the appropriate Feynman-rule tensor. The four-gluon vertex is treated through the auxiliary antisymmetric tensor route, so it is still represented by cubic current combinations.

AMPLICOL’s optimized mode does not loop over full helicity configurations as independent computations. It weaves helicity information into the current table itself. Each external helicity state is a source current with a unique bit in an `ext_cur`-style bitset. An internal current is identified by

$$K(\mathcal{J}) = (t, \chi, \text{bin}(I), \text{ext_cur}), \quad (2)$$

where `bin(I)` records the external subset and `ext_cur` is the union of the source-current bits feeding the current. When a new vertex contribution produces the same key as an existing current, AMPLICOL appends that vertex to the existing current rather than creating a duplicate node. After all currents are constructed, `curr2amp` records the two endpoint currents whose contraction gives each retained helicity amplitude.

The generated Fortran library has the same layered structure:

```

subroutine evaluate_amp(p, amps)
  complex(kind=8) :: val_c(1:6, n_cur)
  complex(kind=8) :: int_c(1:6, n_vert)

  call compute_external_currents(pp, val_c)
  do size = 2, n_external_minus_one
    call compute_vertices_size(pp, val_c, int_c)
    call compute_currents_size(pp, val_c, int_c)
  end do
  call compute_amps(amps, val_c)
end subroutine

```

The arrays `val_c` and `int_c` are the numerical current and interaction tables. This is the performance-critical idea that PYAMPLICOL keeps: one topological sweep produces all retained amplitudes, and shared parent currents are evaluated once.

2 The pyAmpliCol DAG

PYAMPLICOL first builds an explicit Python representation of the same current table. In the generic compiler, a current is identified by a model-driven physics index rather than by a process-family name. The identity used for deduplication is

$$K_{\text{PYAMPLICOL}} = (\text{particle_id}, L, H, \chi, S, F, Q, C, P, A), \quad (3)$$

where L is the sorted tuple of external labels, H is the AMPLICOL-style helicity-source ancestry bitset, χ is chirality, S is any spin-state tag needed by the source or propagator, F is flavour flow, Q is charge flow, C is the leading-color state, P is the momentum mask, and A is an optional auxiliary kind such as the antisymmetric tensor used for the four-gluon vertex. The implementation also carries `ordered_external_labels`. That field is leading-color ordering metadata used by the color engine and amplitude grouping, but it is deliberately not part of equality or hashing. Deduplication is therefore exactly equality of Eq. (3); no rule is allowed to ask whether the process is “ Z plus jets” or any other family.

field	role in the current identity
<code>particle_id</code>	Model/UFO particle id, e.g. 21, 23, 1, -1 , or an auxiliary id.
<code>external_labels</code> and <code>external_mask</code>	Which external legs are contained in the current. The mask is the bit representation of the labels.
<code>helicity_ancestry</code>	Bitwise union of the source-current helicity bits feeding this current. This is how helicity configurations are woven into one shared DAG.
<code>chirality</code> and <code>spin_state</code>	Local spin information, especially for Weyl fermions and massive-vector sources.
<code>flavour_flow</code> and <code>charge_flow</code>	Quantum-number flow checked locally by model vertices, e.g. $u \rightarrow dW^+$ or $e^+e^- \rightarrow \gamma/Z$.
<code>color_state</code>	Colour-ordered open-line or trace state used to build the amplitude vector. LC, NLC and full colour differ in the final sparse colour contraction over this vector.
<code>momentum_mask</code>	Which all-outgoing momenta are summed for propagators and momentum-dependent vertices.
<code>auxiliary_kind</code>	Optional tag distinguishing auxiliary tensor currents from ordinary particles.

Source currents are built from external leg metadata. Internal currents are then generated by increasing current size, with all interactions that feed the same current index attached to one node.

```

for external_helicity_source s:
    add_current(key=(pdg_s, labels_s, chirality_s, bit_s), is_source=True)

for current_size in increasing_sizes:
    for compatible_split A | B:
        result_key = combine_key(key[A], key[B], vertex_kind)
        result = add_or_reuse_current(result_key)
        add_interaction(left=A, right=B, result=result, vertex=vertex_kind)

for retained_helicity_ancestry:
    add_amplitude(left=current_with_Z_and_gluons, right=anti_current)

prune_dead_currents_after_helicity_filter()

```

At runtime, PYAMPLICOL mirrors the Fortran table layout with contiguous arrays:

$$C_{s,i,c} \equiv \text{current_rows}[s, i, c], \quad T_{s,v,c} \equiv \text{interaction_rows}[s, v, c],$$

where s is the phase-space sample in the current batch, i is a current id, v is an interaction id, and c is a component index. External wavefunctions fill source-current rows once per sample. Bounded evaluator blocks then fill interactions and derived currents stage by stage:

```

fill_source_currents(current_rows, momenta, helicity_sources)

for stage in current_size_layers:
    state = ParamBuilder.pack(current_rows, interaction_rows, momenta)
    interaction_rows[:, stage.vertices] = vertex_eval[stage](state)
    current_rows[:, stage.currents] = current_eval[stage](state)

amps = amplitude_eval(ParamBuilder.pack(current_rows, momenta))
me2 = normalization * sum(weight[h] * abs(amps[h])**2 for h in retained_h)

```

This is why the evaluator-only benchmark is meaningful: the expensive recursion is inside compiled evaluator calls, while Python only performs source wavefunction construction, parameter packing, and table transfers.

3 Spenso Tensor Networks and hep_lib

Spenso is used to express Lorentz, spinor, auxiliary-tensor, and eventually color contractions in a representation-aware way. A tensor-network expression contains named tensors with typed slots:

$$\mathcal{V}_{\mu\nu}{}^\rho(p_A, p_B) A^\mu B^\nu, \quad \text{or in code: } V(m("a"), m("b"), m("out")) * A(m("a")) * B(m("b")).$$

The expression is not expanded. It is a compact product of tensor factors with shared representation labels. The library `hep_lib` is the dictionary that tells Spenso what each tensor name means: dense model tensors, metric tensors, momentum-dependent propagators, and parametric rank-one tensors representing runtime inputs.

```
library = model.build_tensor_library()      # returns TensorLibrary.hep_lib_atom()
builder = ParamBuilder()
mink = Representation.mink(4)

builder.register_rank1_tensor(
    library,
    tensor_name="current_A",
    representation=mink,
    head=("stage", "A"),
    length=4,
    role="block_current",
)
builder.register_rank1_tensor(
    library,
    tensor_name="p_A",
    representation=mink,
    head=("stage", "pA"),
    length=4,
    role="block_momentum",
    real_valued=True,
)

raw = V3g(mink("a"), mink("b"), mink("out"), p_A="p_A", p_B="p_B") \
    * TensorName("current_A")(mink("a")).to_expression() \
    * TensorName("current_B")(mink("b")).to_expression()

network = TensorNetwork(raw, library)
network.execute(library=library)
out_tensor = network.result_tensor(library)
evaluator = out_tensor.evaluator(params=builder.parameter_symbols())
```

The important point is that `hep_lib` stays fixed during the execution. New graph currents are not registered as permanent physics tensors. Instead, the current values are runtime parameters for a bounded block. Spenso executes the factorized tensor network, contracts the representation indices, and returns a parametric output tensor. Symbolica then compiles that parametric tensor to an evaluator. For high multiplicity, PYAMPLICOL interleaves this process: after a current or current-size layer is contracted, its numerical output is written back into the current table and becomes an input parameter for later blocks. This avoids one giant scalar expression and keeps the DAG close to AMPLICOL's execution model.

The current NLC/full-colour implementation keeps the kinematic DAG colour-ordered and applies a sparse colour matrix to the amplitude vector. For open-line sectors, including arbitrary many quark lines with gluons distributed on the lines, PYAMPLICOL contracts the product of left colour strings with the conjugated right strings into closed fundamental traces. Those small colour trace expressions are simplified separately from the kinematic DAG; NLC is obtained by keeping the leading and first $1/N_c^2$ -suppressed powers, while full colour keeps the complete sparse metric. If a later implementation puts explicit colour tensors inside a Spenso block, colour simplification still belongs before Spenso execution:

```
raw ← idenso.simplify_color(raw),      then build TensorNetwork(raw, hep_lib).
```

For the present NLC/full-colour milestone, colour algebra is confined to the small colour-basis overlap problem and the Rusticol final contraction, not to expanded kinematic expressions. Fortran AmpliCol is the direct reference for zero, one and two quark-line colour matrices; beyond that, pyAmpliCol's generic open-line metric is validated through raw-amplitude probes and internal colour consistency checks.

4 Example: $d\bar{d} \rightarrow Zgg$

This section spells out one concrete DAG so that the split between compiled current contractions and Rust-interpreted scheduling is explicit. The user-level process is

$$d(p_1) \bar{d}(p_2) \rightarrow Z(p_3) g(p_4) g(p_5).$$

PYAMPLICOL stores the process in the all-outgoing convention. Thus labels $1, \dots, 5$ carry the fields

$$\bar{d}_1, \quad d_2, \quad Z_3, \quad g_4, \quad g_5,$$

with momenta $-p_1, -p_2, p_3, p_4$, and p_5 , respectively. A label set such as $\{2, 4, 5\}$ is represented by $L = (2, 4, 5)$ and by the mask

$$m(2, 4, 5) = 2^{2-1} + 2^{4-1} + 2^{5-1} = 26.$$

The momentum mask is the same bit pattern in this example, and it tells Rusticol which stored momentum sum must be passed to the next compiled evaluator.

For one leading-color sector, the colored ordered labels are

$$O = (2, 4, 5, 1),$$

with the color-singlet Z_3 allowed to attach anywhere on the quark line. The color engine may also provide the sector $O = (2, 5, 4, 1)$. Nothing in the DAG builder special-cases this as a “ $Z + 2g$ ” process: source currents are ordinary external-leg currents and all later currents are discovered from model vertices.

Source current indices

For a fixed retained source helicity choice, the source entries look as follows. The bit b_i denotes the unique helicity-source bit assigned to that external state.

source	current	mask	index payload
S_1	$\mathcal{J}_{\bar{d}}(\{1\})$	1	(pid = -1, $L = (1)$, $H = b_1, \chi, S, F =$ $(-1), Q, C_{\bar{q}}, P = 1)$
S_2	$\mathcal{J}_d(\{2\})$	2	(pid = 1, $L = (2)$, $H = b_2, \chi, S, F =$ $(1), Q, C_q, P = 2)$
S_3	$\epsilon_Z(\{3\})$	4	(pid = 23, $L = (3)$, $H = b_3, S =$ $h_Z, C_{\text{singlet}}, P = 4)$
S_4	$\epsilon_g(\{4\})$	8	(pid = 21, $L = (4)$, $H = b_4, S = h_4, C_g, P = 8)$
S_5	$\epsilon_g(\{5\})$	16	(pid = 21, $L = (5)$, $H = b_5, S = h_5, C_g, P = 16)$

The actual stored index also contains the exact leading-color sector id, line-group metadata, and the current dimension. For this example the quark and anti-quark source currents have Weyl-spinor components, gluons and the Z have Lorentz-vector components, and auxiliary tensor currents have the antisymmetric tensor components required by the cubicized four-gluon rule.

What each compiled stage computes

The generic stage compiler groups interactions by the size of the result current. For each group it emits a multi-output Symbolica evaluator. That evaluator is compiled once and receives only parent-current components, momentum sums, and real couplings as parameters. It returns the components of all result currents in the stage.

For the sector $O = (2, 4, 5, 1)$, the schematic stage contents are:

$$E_2 : \quad \mathcal{J}_g(4, 5) = P_g(p_{45}) \mathcal{V}_{3g}(S_4, S_5; p_4, p_5), \quad (4)$$

$$\mathcal{J}_T(4, 5) = \mathcal{V}_{ggT}(S_4, S_5), \quad (5)$$

$$\mathcal{J}_d(2, 4) = P_q(p_{24}) \mathcal{V}_{ggq}(S_2, S_4; p_2, p_4), \quad (6)$$

$$\mathcal{J}_d(2, 3) = P_q(p_{23}) \mathcal{V}_{qZq}(S_2, S_3; p_2, p_3), \quad (7)$$

plus the analogous allowed currents for the other color sector and for the anti-quark side. The notation E_2 means a compiled evaluator for all two-label result currents, not a Python function that loops over these formulas component by component.

The next stage consumes the rows written by E_2 :

$$E_3 : \quad \mathcal{J}_d(2, 4, 5) = P_q(p_{245}) [\mathcal{V}_{ggq}(\mathcal{J}_d(2, 4), S_5) + \mathcal{V}_{ggq}(S_2, \mathcal{J}_g(4, 5))], \quad (8)$$

$$\mathcal{J}_d(2, 3, 4) = P_q(p_{234}) [\mathcal{V}_{qZq}(\mathcal{J}_d(2, 4), S_3) + \mathcal{V}_{ggq}(\mathcal{J}_d(2, 3), S_4)]. \quad (9)$$

The stage compiler does not expand these expressions into diagrams. It collects all interaction records whose output index is the same current and compiles the sum into the output components of that current. If two different splits produce the same current index, their contributions are accumulated inside the same compiled stage output.

The four-label stage then builds the endpoint current used by the closure:

$$E_4 : \quad \mathcal{J}_d(2, 3, 4, 5) = P_q(p_{2345}) \sum_{\text{compatible splits}} \mathcal{V}(\mathcal{J}_{\text{left}}, \mathcal{J}_{\text{right}}). \quad (10)$$

The sum includes, for example, gluon-first and Z -insertion contributions such as

$$\mathcal{V}_{qZq}(\mathcal{J}_d(2, 4, 5), S_3), \quad \mathcal{V}_{qgq}(\mathcal{J}_d(2, 3, 4), S_5), \quad \mathcal{V}_{qgq}(\mathcal{J}_d(2, 3), \mathcal{J}_g(4, 5)).$$

Those are still local vertex applications selected by the model and color engine. The result is one stored current row, identified by the full CurrentIndex and by its value-slot range in the artifact.

Finally, a compiled amplitude evaluator returns raw complex roots:

$$E_{\text{amp}} : \quad R_a = \mathcal{V}_{\text{close}}(\mathcal{J}_d(2, 3, 4, 5), S_1), \quad (11)$$

with one output for each retained root. Roots with the same $(H, \text{color sector}, O)$ coherent-group key are summed before squaring. The reported leading-color matrix element is

$$|\mathcal{M}|_{\text{LC}}^2 = \mathcal{N}_{\text{AC}} \sum_{\text{groups } g} w_g \left| \sum_{a \in g} R_a \right|^2. \quad (12)$$

The same example as a DAG

A stripped-down adjacency-list representation of this example is:

```
sources interpreted by Rusticol:
S1 = J(pid=-1, labels={1}, H=b1, color=qbar-line)
S2 = J(pid= 1, labels={2}, H=b2, color=q-line)
S3 = J(pid=23, labels={3}, H=b3, color=singlet)
S4 = J(pid=21, labels={4}, H=b4, color=line gluon)
S5 = J(pid=21, labels={5}, H=b5, color=line gluon)

compiled stage E2, subset_size=2:
I0: S4 + S5 -- V3g, Pg --> J(pid=21, labels={4,5})
I1: S4 + S5 -- VggT --> J(aux=T, labels={4,5})
I2: S2 + S4 -- Vqgq,Pq --> J(pid=1, labels={2,4})
I3: S2 + S3 -- VqZq,Pq --> J(pid=1, labels={2,3})

compiled stage E3, subset_size=3:
I4: J(2,4) + S5 -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
I5: S2 + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
I6: J(2,4) + S3 -- VqZq,Pq --> J(pid=1, labels={2,3,4})
I7: J(2,3) + S4 -- Vqgq,Pq --> J(pid=1, labels={2,3,4})

compiled stage E4, subset_size=4:
I8: J(2,4,5) + S3 -- VqZq,Pq --> J(pid=1, labels={2,3,4,5})
I9: J(2,3,4) + S5 -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})
I10: J(2,3) + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})

compiled amplitude stage:
A0: J(pid=1, labels={2,3,4,5}) + S1 -- close --> R0
```

This listing is schematic: the real artifact contains all retained helicity ancestries, both leading-color sectors, source dimensions, output component ranges, momentum-slot ids, couplings, and color weights. The important point is the division of labor. The evaluator E_s performs the tensor contractions and the algebraic sum over interactions in stage s . Rusticol interprets the DAG schedule: fill sources, compute momentum sums, call E_2 , scatter its outputs into current storage, call E_3 , scatter again, and so on until E_{amp} and the final coherent square-sum.

5 Rusticol: the Implemented Rust Runtime

The previous staged PYAMPLICOL runtime proved that the shared-current DAG could match Fortran AMPLICOL numerically, but its Python loop still moved current tables between many evaluator calls. The current implementation

keeps Python as the process generator and validation driver, but moves the hot eager-DAG sweep into a PyO3 extension named `rusticol`. The generated artifact is now a self-contained process directory used by both runtimes:

```
rt_py = pyamplicol.load_process(process_dir, runtime="python")
values_py = rt_py.evaluate(momenta)

import rusticol
rt = rusticol.Runtime.load(process_dir)
values = rt.evaluate(momenta)
profile = rt.profile(momenta)
```

The Python and Rust runtimes read the same `process_manifest.json`, the same Symbolica evaluator manifests, and the same validation momenta. The Python path is kept as a transparent reference implementation. The Rust path is the performance path.

A process directory contains four classes of data. First, it records process and model metadata: external PDG order, momentum conventions, particle and vertex tables, couplings, masses, symmetry factors, and leading-color normalization. Second, it stores the shared-current DAG: source currents, current ids, dimensions, interaction stages, output slots, retained amplitude records, helicity weights, and color factors. Third, it stores the compiled Symbolica evaluator artifacts for each stage and for the final amplitude/raw-sum stage. Fourth, it contains standalone validation assets: `validation_momenta.json` with decimal-string momenta and a `check_standalone.py` script that imports only `rusticol`, plus `numpy` when available for the optimized f64 input path.

```
# Generated by pyAmpliCol.
process_dir/
  process_manifest.json
  manifest.json
  validation_momenta.json
  check_standalone.py
  compiled/           # C++/ASM shared libraries when present
  ... evaluator states ...
```

`rusticol.Runtime.load` validates the schema, checks that the current process family is supported, loads every Symbolica evaluator, computes direct state offsets for stage outputs, allocates reusable scratch buffers, and builds a fixed execution plan. The optimized f64 path accepts NumPy arrays with shape `(batch, n_external, 4)`. For each batch, Rust fills source wavefunctions and momentum sums, executes stage evaluators in topological order, writes the returned current components into the contiguous state table, executes the amplitude/raw-sum evaluator, applies the Fortran-matched normalization, and returns a NumPy array of matrix elements.

```
for point in batch {
    fill_source_currents(state_row, point);
    fill_momentum_sums(state_row, point);
}

for stage in stages {
    evaluated = stage.evaluator.evaluate_batch(state);
    assign_stage_outputs(state, evaluated, stage.output_offsets);
}

raw = amplitude_stage.evaluate_raw_sums(state);
me2 = normalization * raw;
```

The `profile` API reports the same split used in the benchmarks: source fill, momentum setup, stage evaluator time, output assignment, amplitude evaluator time, final reduction, total wall time, and memory snapshots where the platform exposes them.

The precision API follows Symbolica’s model. `evaluate` and `evaluate_with_prec(..., 16)` use the specialized native f64 path. `evaluate_with_prec(..., 32)` maps the serialized exact evaluator state to Symbolica’s `DoubleFloat` coefficients. Other positive precisions map to arbitrary-precision `Float` evaluators. The high-precision routes share the same stage orchestration code through generic Rust traits, while the f64 route remains specialized and avoids high-precision allocation overhead.

The artifact schema is process-generic. The staged execution format no longer needs a Rust-side process-family branch: each process directory supplies the source layout, current slots, stage records, amplitude roots, coherent groups, and evaluator artifacts needed by `Rusticol`. If a model vertex or color rule is not implemented yet, generation fails with an explicit unsupported-kernel diagnostic rather than falling back to a hidden family path.

6 Generic LC Performance Matrix

Each cell compares generated-library `AMPLICOL` against `PYAMPLICOL` staged-DAG JIT O3 and C++ O3 at the same final-state multiplicity n . Cell slots are `AMPLICOL`, `PYAMPLICOL` JIT, and `PYAMPLICOL` C++ O3; each slot shows generation time above runtime per phase-space point. `PYAMPLICOL` runtime multipliers are (wall|core) relative to `AMPLICOL`. If no direct `AMPLICOL` reference exists, completed `PYAMPLICOL` slots show absolute timings. Ratio colours are green below one, orange below two, and red otherwise. The summary rows give min|max|avg|med; multiplier summaries add sum, the ratio of paired summed `PYAMPLICOL` and `AMPLICOL` times. Missing and structural N/A cells are ignored. Validated rows use the same phase-space point and a 10^{-8} relative tolerance.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8} .

ID	base process	n=1					n=2					n=3				
8	1 $d\bar{d} \rightarrow Z + (n-1)g$	2.63 s	(x0.173)	(x0.457)			2.222 s	(x0.263)	(x0.671)			2.259 s	(x0.331)	(x1.09)		
		0.0219 us	(x8.12 x2.17)	(x8.11 x1.97)			0.123 us	(x3.06 x1.61)	(x3.38 x1.88)			0.439 us	(x1.85 x1.24)	(x2.21 x1.59)		
	2 $u\bar{d} \rightarrow W^+ + (n-1)g$	2.167 s	(x0.229)	(x0.417)			2.146 s	(x0.262)	(x0.619)			2.22 s	(x0.325)	(x0.841)		
		0.012 us	(x12.5 x3.15)	(x12.4 x2.64)			0.0655 us	(x4.41 x2.11)	(x4.60 x2.19)			0.244 us	(x2.44 x1.56)	(x2.70 x1.82)		
	3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A					2.217 s	(x0.242)	(x0.6)			2.248 s	(x0.312)	(x0.782)		
							0.0652 us	(x3.84 x1.70)	(x4.12 x1.93)			0.172 us	(x2.54 x1.45)	(x2.71 x1.68)		
	4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A					2.158 s	(x0.252)	(x0.568)			2.575 s	(x0.286)	(x0.584)		
							0.0317 us	(x6.00 x2.35)	(x6.09 x2.21)			0.0628 us	(x4.64 x2.26)	(x4.39 x2.04)		
	5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A					2.269 s	(x0.282)	(x0.685)			2.426 s	(x0.342)	(x1.31)		
							0.16 us	(x2.85 x1.51)	(x3.06 x1.75)			0.727 us	(x1.52 x1.05)	(x2.00 x1.49)		
	6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A					2.243 s	(x0.291)	(x0.644)			2.593 s	(x0.327)	(x1.32)		
							0.103 us	(x4.19 x2.20)	(x4.53 x2.54)			0.693 us	(x1.63 x1.14)	(x2.14 x1.62)		
	7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A					2.174 s	(x0.242)	(x0.523)			2.276 s	(x0.336)	(x1.01)		
							0.0484 us	(x5.77 x2.11)	(x5.54 x1.91)			0.242 us	(x2.41 x1.41)	(x2.74 x1.71)		
	8 $gg \rightarrow gg + (n-2)g$	N/A					2.191 s	(x0.261)	(x0.639)			2.301 s	(x0.376)	(x1.78)		
							0.176 us	(x2.02 x1.14)	(x2.30 x1.42)			0.881 us	(x1.29 x0.992)	(x1.87 x1.55)		
	9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A					N/A					2.757 s	(x0.466)	(x2.47)		
												1.54 us	(x1.48 x1.12)	(x2.06 x1.67)		
	10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A					N/A					N/A				
	11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A					N/A					N/A				
	12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A					N/A					N/A				
	13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A					N/A					N/A				
	14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A					N/A				
summary: gen		2.17	2.63	2.4	2.4		2.15	2.27	2.2	2.2		2.22	2.76	2.41	2.3	
		x0.173	x0.229	x0.201	x0.201	x0.199	x0.242	x0.291	x0.262	x0.261	x0.262	x0.286	x0.466	x0.345	x0.331	x0.346
summary: run		0.012	0.0219	0.0169	0.0169		0.0317	0.176	0.0966	0.0842		0.0628	1.54	0.556	0.439	
		x8.12	x12.5	x10.3	x10.3	x9.67	x2.02	x6.00	x4.02	x4.02	x3.40	x1.29	x4.64	x2.20	x1.85	x1.67

Generic LC Performance Matrix (continued)

ID	base process	n=4					n=5					n=6				
6	1 $d\bar{d} \rightarrow Z + (n-1)g$	3.771 s	(x0.601)	(x1.88)			5.75 s	(x0.709)	(x3.30)			4.383 s	(x1.20)	(N/A)		
		1.822 us	(x2.02 x1.53)	(x1.58 x1.30)			8.542 us	(x0.86 x0.702)	(x1.03 x0.883)			13.95 us	(x1.38 x1.21)	(N/A N/A)		
	2 $u\bar{d} \rightarrow W^+ + (n-1)g$	5.569 s	(x0.2)	(x0.826)			2.764 s	(x0.666)	(x4.31)			2.861 s	(x1.38)	(N/A)		
		0.947 us	(x1.53 x1.16)	(x1.96 x1.57)			3.029 us	(x1.39 x1.16)	(x1.83 x1.61)			9.408 us	(x3.72 x3.25)	(N/A N/A)		
	3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	2.433 s	(x0.41)	(x1.32)			2.695 s	(x0.473)	(x2.22)			6.576 s	(x0.748)	(N/A)		
		0.461 us	(x2.08 x1.44)	(x2.29 x1.73)			1.393 us	(x1.45 x1.15)	(x1.89 x1.59)			7.714 us	(x2.00 x1.66)	(N/A N/A)		
	4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	6.057 s	(x0.33)	(x0.371)			2.564 s	(x0.43)	(x1.40)			6.168 s	(x0.559)	(N/A)		
		0.514 us	(x2.21 x1.36)	(x1.06 x0.69)			0.65 us	(x1.94 x1.49)	(x2.26 x1.82)			5.988 us	(x1.47 x1.21)	(N/A N/A)		
	5 $d\bar{d} \rightarrow ZZ + (n-2)g$	5.774 s	(x0.537)	(x1.46)			3.18 s	(x1.08)	(x6.67)			9.352 s	(x1.58)	(N/A)		
		4.891 us	(x1.47 x1.13)	(x0.825 x0.686)			6.634 us	(x2.51 x2.07)	(x1.99 x1.69)			44.21 us	(x1.52 x1.36)	(N/A N/A)		
	6 $gg \rightarrow t\bar{t} + (n-2)g$	9.799 s	(x0.329)	(x1.06)			14.52 s	(x0.237)	(x2.39)			77.62 s	(x0.184)	(N/A)		
		6.155 us	(x1.49 x1.16)	(x0.776 x0.654)			8.423 us	(x1.50 x1.28)	(x2.55 x2.31)			54.88 us	(x1.85 x1.70)	(N/A N/A)		
	7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	6.525 s	(x0.302)	(x0.649)			4.146 s	(x0.628)	(x3.13)			12.95 s	(x0.595)	(N/A)		
		1.017 us	(x3.61 x2.56)	(x1.87 x1.48)			3.885 us	(x2.39 x1.93)	(x1.53 x1.34)			23.77 us	(x1.06 x0.927)	(N/A N/A)		
	8 $gg \rightarrow gg + (n-2)g$	7.364 s	(x0.58)	(x1.77)			7.899 s	(x0.952)	(x5.57)			5.603 s	(x1.45)	(N/A)		
		8.457 us	(x1.62 x1.33)	(x0.713 x0.63)			16.87 us	(x1.89 x1.69)	(x1.81 x1.68)			32.56 us	(x1.94 x1.82)	(N/A N/A)		
	9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	8.47 s	(x0.672)	(x2.09)			13.75 s	(x0.986)	(x3.29)			9.91 s	(x1.32)	(N/A)		
		8.798 us	(x2.25 x1.84)	(x0.959 x0.813)			36.34 us	(x1.43 x1.26)	(x0.585 x0.525)			35.84 us	(x2.22 x2.06)	(N/A N/A)		
	10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	11.66 s	(x0.184)	(x0.399)			15.3 s	(x0.137)	(x0.632)			10.85 s	(x0.234)	(N/A)		
		2.057 us	(x2.18 x1.61)	(x0.973 x0.791)			5.502 us	(x0.615 x0.509)	(x0.745 x0.647)			4.005 us	(x3.82 x3.20)	(N/A N/A)		
	11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	9.043 s	(x0.233)	(x0.575)			6.269 s	(x0.378)	(x2.62)			20.03 s	(x0.272)	(N/A)		
		1.731 us	(x2.89 x2.12)	(x1.38 x1.13)			3.492 us	(x1.60 x1.35)	(x2.07 x1.81)			13.62 us	(x1.41 x1.24)	(N/A N/A)		
	12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	12 s	(x0.193)	(x0.42)			10.31 s	(x0.344)	(x0.94)			6.737 s	(x0.354)	(N/A)		
		3.007 us	(x1.07 x0.805)	(x0.69 x0.575)			6.45 us	(x0.672 x0.542)	(x0.667 x0.569)			6.802 us	(x0.815 x0.704)	(N/A N/A)		
	13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	5.597 s	(x0.304)	(x0.363)			8.779 s	(x0.253)	(x0.419)			6.578 s	(x0.247)	(N/A)		
		0.186 us	(x5.85 x3.35)	(x2.45 x1.42)			1.998 us	(x1.27 x0.878)	(x0.595 x0.447)			2.384 us	(x0.989 x0.805)	(N/A N/A)		
	14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A					N/A	5.213 s	N/A		
												N/A	(15.1 11.7)	N/A		
summary: gen		2.43	12	7.24	6.52		2.56	15.3	7.53	6.27		2.86	77.6	13.8	6.74	
		x0.184	x0.672	x0.375	x0.329	x0.35	x0.137	x1.08	x0.559	x0.473	x0.501	x0.184	x1.58	x0.779	x0.595	x0.488
summary: run		0.186	8.8	3.08	1.82		0.65	36.3	7.94	5.5		2.38	54.9	19.6	13.6	
		x1.07	x5.85	x2.33	x2.08	x1.86	x0.615	x2.51	x1.50	x1.45	x1.48	x0.815	x3.82	x1.86	x1.52	x1.79

Generic LC Performance Matrix (continued)

ID	base process	n=7					n=8					n=9			
1	$d\bar{d} \rightarrow Z + (n-1)g$	5.267 s	(x1.38)	(N/A)			9.814 s	(x1.81)	(N/A)			25.27 s	(N/A)	(N/A)	
		38.61 us	(x1.95 x1.81)	(N/A N/A)			93.49 us	(x2.03 x1.98)	(N/A N/A)			227 us	(N/A N/A)	(N/A N/A)	
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	3.825 s	(x1.19)	(N/A)			5.156 s	(x2.21)	(N/A)			N/A			
		29.16 us	(x1.64 x1.52)	(N/A N/A)			70.68 us	(x2.07 x1.99)	(N/A N/A)						
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	3.113 s	(x0.864)	(N/A)			4.218 s	(x1.48)	(N/A)			N/A			
		12.65 us	(x1.15 x1.00)	(N/A N/A)			33.47 us	(x1.56 x1.44)	(N/A N/A)						
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	2.777 s	(x0.505)	(N/A)			2.975 s	(x1.23)	(N/A)			N/A			
		7.223 us	(x1.33 x1.18)	(N/A N/A)			21.68 us	(x1.56 x1.43)	(N/A N/A)						
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	7.223 s	(x1.52)	(N/A)			17.91 s	(x1.53)	(N/A)			N/A			
		47.72 us	(x1.50 x1.41)	(N/A N/A)			109 us	(x2.17 x2.10)	(N/A N/A)						
6	$gg \rightarrow t\bar{t} + (n-2)g$	255 s	(x0.038)	(N/A)			N/A					N/A			
		61.69 us	(x2.31 x2.24)	(N/A N/A)											
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	13.27 s	(x0.373)	(N/A)			N/A					N/A			
		28.34 us	(x2.19 x2.05)	(N/A N/A)											
8	$gg \rightarrow gg + (n-2)g$	12.58 s	(x0.921)	(N/A)			N/A					N/A			
		86.07 us	(x2.02 x1.97)	(N/A N/A)											
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	24.51 s	(x0.99)	(N/A)			N/A					N/A			
		83.77 us	(x1.93 x1.85)	(N/A N/A)											
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	8.922 s	(x0.336)	(N/A)			N/A					N/A			
		10.29 us	(x1.46 x1.25)	(N/A N/A)											
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	40.24 s	(x0.182)	(N/A)			N/A					N/A			
		33.09 us	(x2.16 x2.02)	(N/A N/A)											
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	10.52 s	(x0.259)	(N/A)			N/A					N/A			
		16.25 us	(x0.731 x0.633)	(N/A N/A)											
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	14.45 s	(x0.109)	(N/A)			N/A					N/A			
		7.626 us	(x0.829 x0.727)	(N/A N/A)											
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	31.22 s	N/A			N/A					N/A			
		N/A	(317 304)	N/A											
summary: gen		2.78	255	30.9	10.5		2.98	17.9	8.01	5.16		25.3	25.3	25.3	25.3
		x0.038	x1.52	x0.667	x0.505	x0.229	x1.23	x2.21	x1.65	x1.53	x1.66	N/A			
summary: run		7.22	86.1	35.6	29.2		21.7	109	65.6	70.7		227	227	227	227
		x0.731	x2.31	x1.63	x1.64	x1.87	x1.56	x2.17	x1.88	x2.03	x2.00	N/A			

Matrix run settings. Generated by docs/result_matrix.py with -color-accuracy 1c, from result_matrix_data.json into result_matrix_table.tex. AMPLICOL uses `amplicol_generate -library=create,make amplicol_generate_library`, and the generated-library -library=use benchmark. PYAMPLICOL uses Rusticol double precision, batch size 64, output chunk size 128, stage-local evaluator inputs, ten Horner iterations, Symbolica's default CPE iteration choice, and enlarged Horner/common-pair limits. JIT rows use SymJIT O3; timing runs use -target-runtime 10. Only C++ O3 generation is subject to the 15-minute compile cap.

Structural reference limitations. The four-quark-line row $d\bar{d} \rightarrow u\bar{u} s\bar{s} c\bar{c} + (n-6)g$, is a structural AMPLICOL N/A: the current Fortran path stops above its supported quark-line count. Completed PYAMPLICOL cells are shown as absolute timings.

C++ O3 compile-budget gaps. These cells have completed AMPLICOL and PYAMPLICOL JIT rows but no completed C++ O3 row: $d\bar{d} \rightarrow Z + (n-1)g$, $n=6$: not run; $d\bar{d} \rightarrow Z + (n-1)g$, $n=7$: not run; $d\bar{d} \rightarrow Z + (n-1)g$, $n=8$: not run; $u\bar{d} \rightarrow W^+ + (n-1)g$, $n=6$: not run; $u\bar{d} \rightarrow W^+ + (n-1)g$, $n=7$: not run; $u\bar{d} \rightarrow W^+ + (n-1)g$, $n=8$: not run; $d\bar{d} \rightarrow e^+e^- + (n-2)g$, $n=6$: not run; $d\bar{d} \rightarrow e^+e^- + (n-2)g$, $n=7$: not run; plus 23 more localized entries.

7 Dedicated $d\bar{d} \rightarrow Z + \text{Gluon}$ Performance

n	process	route	setup	gen [s]	wall [us/pt]	eval [us/pt]	notes
1	$d\bar{d} \rightarrow Z + (1-1)*g$	reference	AMPLiCol	2.63	N/A	0.0219	
1	$d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	PYAMPLiCol JIT O1	0.413 (x0.16)	0.226 (x10.3)	0.0611 (x2.79)	
1	$d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	PYAMPLiCol ASM O3	1.47 (x0.56)	0.268 (x12.2)	0.107 (x4.91)	
1	$d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	PYAMPLiCol C++ O3	1.38 (x0.52)	0.217 (x9.89)	0.054 (x2.46)	
1	$d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	PYAMPLiCol JIT O3	0.456 (x0.17)	0.178 (x8.12)	0.0475 (x2.17)	
2	$d\bar{d} \rightarrow Z + (2-1)*g$	reference	AMPLiCol	2.22	N/A	0.123	
2	$d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	PYAMPLiCol JIT O1	0.433 (x0.20)	0.462 (x3.75)	0.24 (x1.95)	
2	$d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	PYAMPLiCol ASM O3	1.76 (x0.79)	0.815 (x6.61)	0.602 (x4.89)	
2	$d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	PYAMPLiCol C++ O3	1.93 (x0.87)	0.438 (x3.55)	0.244 (x1.98)	
2	$d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	PYAMPLiCol JIT O3	0.585 (x0.26)	0.377 (x3.06)	0.198 (x1.61)	
3	$d\bar{d} \rightarrow Z + (3-1)*g$	reference	AMPLiCol	2.26	N/A	0.439	
3	$d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	PYAMPLiCol JIT O1	0.426 (x0.19)	0.839 (x1.91)	0.559 (x1.27)	
3	$d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	PYAMPLiCol ASM O3	1.53 (x0.68)	2.05 (x4.67)	1.76 (x4.00)	
3	$d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	PYAMPLiCol C++ O3	2.54 (x1.13)	1.13 (x2.56)	0.81 (x1.85)	
3	$d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	PYAMPLiCol JIT O3	0.749 (x0.33)	0.813 (x1.85)	0.544 (x1.24)	
4	$d\bar{d} \rightarrow Z + (4-1)*g$	reference	AMPLiCol	3.77	N/A	1.82	
4	$d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	PYAMPLiCol JIT O1	0.741 (x0.20)	2.14 (x1.18)	1.66 (x0.91)	
4	$d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	PYAMPLiCol ASM O3	2.57 (x0.68)	5.95 (x3.27)	5.42 (x2.98)	
4	$d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	PYAMPLiCol C++ O3	6.89 (x1.83)	2.83 (x1.55)	2.35 (x1.29)	
4	$d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	PYAMPLiCol JIT O3	2.27 (x0.60)	3.68 (x2.02)	2.79 (x1.53)	
5	$d\bar{d} \rightarrow Z + (5-1)*g$	reference	AMPLiCol	5.75	N/A	8.54	
5	$d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	PYAMPLiCol JIT O1	1.23 (x0.21)	6.66 (x0.78)	5.51 (x0.64)	
5	$d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	PYAMPLiCol ASM O3	4.59 (x0.80)	19.5 (x2.28)	17.8 (x2.09)	
5	$d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	PYAMPLiCol C++ O3	18 (x3.13)	8.62 (x1.01)	7.51 (x0.88)	
5	$d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	PYAMPLiCol JIT O3	4.08 (x0.71)	7.35 (x0.86)	5.99 (x0.70)	
6	$d\bar{d} \rightarrow Z + (6-1)*g$	reference	AMPLiCol	4.38	N/A	14	
6	$d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	PYAMPLiCol JIT O1	2.77 (x0.63)	20.3 (x1.45)	17.7 (x1.27)	
6	$d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	PYAMPLiCol ASM O3	10.6 (x2.41)	56.6 (x4.06)	53.1 (x3.81)	
6	$d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	PYAMPLiCol C++ O3	60.7 (x13.9)	36.6 (x2.62)	33.2 (x2.38)	
6	$d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	PYAMPLiCol JIT O3	5.26 (x1.20)	19.3 (x1.38)	16.9 (x1.21)	
7	$d\bar{d} \rightarrow Z + (7-1)*g$	reference	AMPLiCol	5.27	N/A	38.6	
7	$d\bar{d} \rightarrow Z + (7-1)*g$	Rusticol	PYAMPLiCol JIT O1	7.33 (x1.39)	62.7 (x1.62)	57.9 (x1.50)	
7	$d\bar{d} \rightarrow Z + (7-1)*g$	Rusticol	PYAMPLiCol ASM O3	20.3 (x3.86)	146 (x3.77)	139 (x3.61)	
7	$d\bar{d} \rightarrow Z + (7-1)*g$	Rusticol	PYAMPLiCol C++ O3	N/A	N/A	N/A	
7	$d\bar{d} \rightarrow Z + (7-1)*g$	Rusticol	PYAMPLiCol JIT O3	7.28 (x1.38)	75.3 (x1.95)	69.9 (x1.81)	
8	$d\bar{d} \rightarrow Z + (8-1)*g$	reference	AMPLiCol	9.81	N/A	93.5	
8	$d\bar{d} \rightarrow Z + (8-1)*g$	Rusticol	PYAMPLiCol JIT O1	18.7 (x1.91)	202 (x2.16)	195 (x2.09)	
8	$d\bar{d} \rightarrow Z + (8-1)*g$	Rusticol	PYAMPLiCol ASM O3	48 (x4.89)	355 (x3.79)	347 (x3.71)	
8	$d\bar{d} \rightarrow Z + (8-1)*g$	Rusticol	PYAMPLiCol C++ O3	N/A	N/A	N/A	
8	$d\bar{d} \rightarrow Z + (8-1)*g$	Rusticol	PYAMPLiCol JIT O3	17.8 (x1.81)	190 (x2.03)	185 (x1.98)	
9	$d\bar{d} \rightarrow Z + (9-1)*g$	reference	AMPLiCol	25.3	N/A	227	
9	$d\bar{d} \rightarrow Z + (9-1)*g$	Rusticol	PYAMPLiCol JIT O1	44.1 (x1.74)	535 (x2.36)	523 (x2.30)	
9	$d\bar{d} \rightarrow Z + (9-1)*g$	Rusticol	PYAMPLiCol ASM O3	109 (x4.32)	1.02e3 (x4.50)	1e3 (x4.43)	
9	$d\bar{d} \rightarrow Z + (9-1)*g$	Rusticol	PYAMPLiCol C++ O3	N/A	N/A	N/A	
9	$d\bar{d} \rightarrow Z + (9-1)*g$	Rusticol	PYAMPLiCol JIT O3	48.2 (x1.91)	609 (x2.68)	595 (x2.62)	

Final-state multiplicity n means $d\bar{d} \rightarrow Z + (n-1)g$. PYAMPLiCol rows use Rusticol, batch size 64, output chunk size 128, stage-local evaluator inputs, ten Horner iterations, and `time-process -target-runtime 10`. The green row is the default PYAMPLiCol route, SymJIT O3. Generated processes are kept under `docs/.z_performance_outputs`; the driver is `docs/run_z_performance_table.py`.

To reproduce one PYAMPLiCol row, run generation followed by timing. This example is the $n = 7$ JIT O3 row; use optimization level 1 for the JIT O1 row, or the compiled/ASM flags from `run_z_performance_table.py` for the ASM row.

```

env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol generate-process 'd d~ > z g g g g g' docs/.z_performance_outputs/manual/n7/jit_o3 --replace --n_cores 5 --color-accuracy lc --batch-size 64 --symbolica-n_cores 5
--symbolica-output-chunk-size 128 --symbolica-stage-local-parameter-layout --symbolica-iterations 10 --symbolica-max-horner-scheme-variables 1000 --symbolica-max-common-pair-cache-entries 5000000 --symbolica-max-common-pair-distance 1000
--lc-sector-ids 0 --reference-color-order 2,4,5,6,7,8,9,1,3 --symbolica-evaluator-backend jit --symbolica-jit-optimization-level 3
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol time-process --target-runtime 10 --batch-size 64 --json docs/.z_performance_outputs/manual/n7/jit_o3

```

8 Generic NLC Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AMPLiCOL, PYAMPLiCOL JIT O3, and PYAMPLiCOL C++ O3. AMPLiCOL uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1					n=2					n=3				
1	$d\bar{d} \rightarrow Z + (n-1)g$	2.625 s	(x0.091)	(x0.392)			2.093 s	(x0.141)	(x0.829)			2.362 s	(x0.149)	(x2.01)		
		11.59 us	(x0.017 x0.005)	(x0.018 x0.005)			25.16 us	(x0.015 x0.008)	(x0.017 x0.009)			90.51 us	(x0.019 x0.012)	(x0.026 x0.019)		
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	1.996 s	(x0.119)	(x0.498)			2.059 s	(x0.107)	(x0.743)			2.286 s	(x0.131)	(x1.49)		
		10.57 us	(x0.016 x0.004)	(x0.016 x0.004)			23.42 us	(x0.013 x0.006)	(x0.013 x0.006)			72.89 us	(x0.015 x0.009)	(x0.019 x0.013)		
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A					2.634 s	(x0.137)	(x0.567)			2.35 s	(x0.118)	(x1.19)		
							16.54 us	(x0.016 x0.007)	(x0.017 x0.008)			38.39 us	(x0.011 x0.006)	(x0.014 x0.009)		
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A					2.266 s	(x0.111)	(x0.62)			2.316 s	(x0.111)	(x1.16)		
							16.37 us	(x0.012 x0.004)	(x0.012 x0.004)			33.39 us	(x0.009 x0.004)	(x0.01 x0.004)		
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A					2.27 s	(x0.104)	(x0.755)			2.454 s	(x0.164)	(x1.82)		
							38.67 us	(x0.012 x0.006)	(x0.014 x0.008)			118 us	(x0.009 x0.006)	(x0.014 x0.01)		
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A					2.269 s	(x0.111)	(x0.985)			2.633 s	(x0.349)	(x7.24)		
							21.5 us	(x0.038 x0.022)	(x0.047 x0.029)			185 us	(x0.058 x0.045)	(x0.064 x0.047)		
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A					2.252 s	(x0.103)	(x0.715)			2.548 s	(x0.156)	(x1.87)		
							18.61 us	(x0.027 x0.011)	(x0.031 x0.013)			75.41 us	(x0.035 x0.024)	(x0.043 x0.03)		
8	$gg \rightarrow gg + (n-2)g$	N/A					2.441 s	(x0.131)	(x2.09)			3.423 s	(x0.686)	(x23.6)		
							54.85 us	(x0.029 x0.02)	(x0.043 x0.033)			1.31e+03 us	(x0.03 x0.027)	(x0.033 x0.028)		
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A					N/A					2.784 s	(x0.192)	(x2.23)		
												268 us	(x0.008 x0.006)	(x0.011 x0.009)		
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A					N/A					N/A				
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A					N/A					N/A				
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A					N/A					N/A				
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A					N/A					N/A				
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A					N/A				
summary: gen		2	2.62	2.31	2.31		2.06	2.63	2.29	2.27		2.29	3.42	2.57	2.45	
		x0.091	x0.119	x0.105	x0.105	x0.103	x0.103	x0.141	x0.118	x0.111	x0.119	x0.111	x0.686	x0.228	x0.156	x0.25
summary: run		10.6	11.6	11.1	11.1		16.4	54.9	26.9	22.5		33.4	1.31e3	243	90.5	
		x0.016	x0.017	x0.016	x0.016	x0.016	x0.012	x0.038	x0.02	x0.016	x0.021	x0.008	x0.058	x0.022	x0.015	x0.027

Generic NLC Performance Matrix (continued)

ID	base process	n=4				n=5				n=6						
1	$d\bar{d} \rightarrow Z + (n-1)g$	2.814 s	(x0.569)	(x11.8)		4.778 s	(x3.56)	(N/A)		32.71 s	(x20.1)	(N/A)				
		990 us	(x0.024 x0.019)	(x0.02 x0.017)		2.27e+04 us	(x0.023 x0.022)	(N/A N/A)		7.62e+05 us	(x0.062 x0.061)	(N/A N/A)				
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	2.609 s	(x0.431)	(x8.26)		3.668 s	(x3.01)	(N/A)		17.32 s	(x23.8)	(N/A)				
		641 us	(x0.019 x0.015)	(x0.022 x0.018)		1.48e+04 us	(x0.021 x0.02)	(N/A N/A)		4.58e+05 us	(x0.058 x0.057)	(N/A N/A)				
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	2.447 s	(x0.181)	(x1.70)		3.172 s	(x0.427)	(N/A)		9.051 s	(x1.61)	(N/A)				
		163 us	(x0.009 x0.007)	(x0.012 x0.009)		1.86e+03 us	(x0.008 x0.006)	(N/A N/A)		4.11e+04 us	(x0.01 x0.009)	(N/A N/A)				
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	2.368 s	(x0.142)	(x0.991)		2.709 s	(x0.261)	(N/A)		6.321 s	(x1.21)	(N/A)				
		104 us	(x0.008 x0.005)	(x0.009 x0.006)		969 us	(x0.007 x0.006)	(N/A N/A)		2.28e+04 us	(x0.009 x0.009)	(N/A N/A)				
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	2.593 s	(x0.371)	(x6.10)		4.15 s	(x1.28)	(N/A)		19.31 s	(x3.40)	(N/A)				
		778 us	(x0.009 x0.007)	(x0.011 x0.009)		1.17e+04 us	(x0.008 x0.007)	(N/A N/A)		2.55e+05 us	(x0.012 x0.011)	(N/A N/A)				
6	$gg \rightarrow t\bar{t} + (n-2)g$	4.661 s	(x2.13)	(x59.3)		27.15 s	(x10.8)	(N/A)		506 s	(N/A)	(N/A)				
		4.52e+03 us	(x0.072 x0.068)	(x0.043 x0.038)		1.43e+05 us	(x0.165 x0.162)	(N/A N/A)		6.3e+06 us	(N/A N/A)	(N/A N/A)				
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	4.168 s	(x0.57)	(x13.7)		16.76 s	(x1.93)	(N/A)		181 s	(N/A)	(N/A)				
		949 us	(x0.05 x0.045)	(x0.042 x0.034)		2.29e+04 us	(x0.07 x0.067)	(N/A N/A)		7.56e+05 us	(N/A N/A)	(N/A N/A)				
8	$gg \rightarrow gg + (n-2)g$	14.39 s	(x2.66)	t/o >15 min		161 s	(N/A)	(N/A)		N/A	N/A	N/A				
		4.54e+04 us	(x0.036 x0.034)	(N/A N/A)		2.05e+06 us	(N/A N/A)	(N/A N/A)		N/A	N/A	N/A				
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	3.067 s	(x0.281)	(x5.95)		6.623 s	(x0.511)	(N/A)		30.84 s	(x0.918)	(N/A)				
		1.25e+03 us	(x0.005 x0.004)	(x0.007 x0.006)		1.2e+04 us	(x0.003 x0.003)	(N/A N/A)		1.88e+05 us	(x0.004 x0.004)	(N/A N/A)				
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	3.928 s	(x0.084)	(x1.24)		4.681 s	(x0.099)	(N/A)		10.74 s	(x0.136)	(N/A)				
		102 us	(x0.016 x0.012)	(x0.02 x0.016)		359 us	(x0.008 x0.007)	(N/A N/A)		2.98e+03 us	(x0.004 x0.004)	(N/A N/A)				
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	2.734 s	(x0.185)	(x4.09)		4.235 s	(x0.502)	(N/A)		17.09 s	(x1.17)	(N/A)				
		188 us	(x0.022 x0.017)	(x0.03 x0.025)		1.8e+03 us	(x0.015 x0.014)	(N/A N/A)		3.05e+04 us	(x0.021 x0.02)	(N/A N/A)				
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	4.068 s	(x0.081)	(x1.37)		5.149 s	(x0.09)	(N/A)		12.47 s	(x0.105)	(N/A)				
		223 us	(x0.007 x0.005)	(x0.01 x0.008)		830 us	(x0.003 x0.003)	(N/A N/A)		6.67e+03 us	(x0.002 x0.001)	(N/A N/A)				
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	0.434 s	5.284 s		N/A	2.59 s	N/A		N/A	43.14 s	N/A				
		N/A	(3.22 2.15)	(3.9 2.38)		N/A	(52.3 41.1)	N/A		N/A	(2e3 1.93e3)	N/A				
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A				N/A				N/A	11.44 s	N/A				
										N/A	(273 226)	N/A				
summary: gen		2.37	14.4	4.15	2.94	2.71	161	20.3	4.73	6.32	506	76.6	17.3			
		x0.081	x2.66	x0.641	x0.326	x1.15	x0.09	x10.8	x2.04	x0.511	x4.43	x0.105	x23.8	x5.84	x1.21	x7.77
summary: run		102	4.54e4	4.61e3	710	359	2.05e6	1.9e5	1.18e4	2.98e3	6.3e6	8.03e5	1.88e5			
		x0.005	x0.072	x0.023	x0.018	x0.038	x0.003	x0.165	x0.03	x0.008	x0.112	x0.002	x0.062	x0.02	x0.01	x0.044

Each cell compares raw generated-library AMPLiCOL colour probes against PYAMPLiCOL staged-DAG JIT O3 and C++ O3 at the same final-state multiplicity n . Cell slots are AMPLiCOL, PYAMPLiCOL JIT, and PYAMPLiCOL C++ O3; each slot shows generation time above runtime per phase-space point. PYAMPLiCOL runtime multipliers are (wall|core) relative to AMPLiCOL. If no direct AMPLiCOL reference exists, completed PYAMPLiCOL slots show absolute timings. For NLC/full-colour tables, AMPLiCOL reference values come from the dedicated raw generated-library colour driver `amplicol_color_library_probe`. Ratio colours are green below one, orange below two, and red otherwise. The summary rows give min|max|avg|med; multiplier summaries add sum, the ratio of paired summed PYAMPLiCOL and AMPLiCOL times. Missing and structural N/A cells are ignored. Validated rows use the same phase-space point and a 10^{-8} relative tolerance.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8} .

Matrix run settings. Generated by `docs/result_matrix.py` with `-color-accuracy nlc`, from `result_matrix_nlc_data.json` into `result_matrix_nlc_table.tex`. AMPLiCOL uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`, preserving NLC/full colour-order interferences. PYAMPLiCOL uses Rusticol double precision, batch size 64, output chunk size 128, stage-local evaluator inputs, ten Horner iterations, Symbolica's default CPE iteration choice, and enlarged Horner/common-pair limits. JIT rows use SymJIT O3; timing runs use `-target-runtime 10`. Only C++ O3 generation is subject to the 15-minute compile cap.

Structural reference limitations. The four-quark-line row $d\bar{d} \rightarrow u\bar{u} s\bar{s} c\bar{c} + (n-6)g$, is a structural AMPLiCOL N/A: the current Fortran path stops above its supported quark-line count. Completed PYAMPLiCOL cells are shown as absolute timings.

Fortran colour-reference limitations. PYAMPLiCOL generated these cells, but the current Fortran colour-reference path stops above two quark lines: $d\bar{d} \rightarrow u\bar{u} s\bar{s} + (n-4)g$, $n=4$; $d\bar{d} \rightarrow u\bar{u} s\bar{s} + (n-4)g$, $n=5$; $d\bar{d} \rightarrow u\bar{u} s\bar{s} + (n-4)g$, $n=6$.

C++ O3 compile-budget gaps. These cells have completed AMPLiCOL and PYAMPLiCOL JIT rows but no completed C++ O3 row: $d\bar{d} \rightarrow Z + (n-1)g$, $n=5$: not run; $d\bar{d} \rightarrow Z + (n-1)g$, $n=6$: not run; $u\bar{d} \rightarrow W^+ + (n-1)g$, $n=5$: not run; $u\bar{d} \rightarrow W^+ + (n-1)g$, $n=6$: not run; $d\bar{d} \rightarrow e^+e^- + (n-2)g$, $n=5$: not run; $d\bar{d} \rightarrow e^+e^- + (n-2)g$, $n=6$: not run; $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$, $n=5$: not run; $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$, $n=6$: not run; plus 13 more localized entries.

9 Generic Full-Colour Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AMPLICOL, PYAMPLICOL JIT O3, and PYAMPLICOL C++ O3. AMPLICOL uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1					n=2					n=3				
1	$d\bar{d} \rightarrow Z + (n-1)g$	2.624 s	(x0.076)	(N/A)			2.314 s	(x0.11)	(N/A)			2.373 s	(x0.165)	(N/A)		
		11.39 us	(x0.017 x0.005)	(N/A N/A)			25.77 us	(x0.015 x0.008)	(N/A N/A)			90.54 us	(x0.019 x0.012)	(N/A N/A)		
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	2.222 s	(x0.089)	(N/A)			2.275 s	(x0.115)	(N/A)			2.376 s	(x0.143)	(N/A)		
		11.55 us	(x0.014 x0.004)	(N/A N/A)			23.87 us	(x0.012 x0.006)	(N/A N/A)			71.14 us	(x0.015 x0.01)	(N/A N/A)		
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A					2.313 s	(x0.114)	(N/A)			2.428 s	(x0.119)	(N/A)		
							16.61 us	(x0.016 x0.007)	(N/A N/A)			39.54 us	(x0.011 x0.006)	(N/A N/A)		
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A					3.28 s	(x0.086)	(N/A)			2.354 s	(x0.115)	(N/A)		
							16.37 us	(x0.012 x0.005)	(N/A N/A)			32.81 us	(x0.009 x0.004)	(N/A N/A)		
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A					2.497 s	(x0.127)	(N/A)			2.412 s	(x0.15)	(N/A)		
							39.49 us	(x0.012 x0.006)	(N/A N/A)			119 us	(x0.01 x0.006)	(N/A N/A)		
6	$g\bar{g} \rightarrow t\bar{t} + (n-2)g$	N/A					2.329 s	(x0.103)	(N/A)			2.63 s	(x0.348)	(N/A)		
							22.03 us	(x0.04 x0.023)	(N/A N/A)			188 us	(x0.057 x0.043)	(N/A N/A)		
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A					2.649 s	(x0.114)	(N/A)			2.578 s	(x0.169)	(N/A)		
							19.24 us	(x0.027 x0.011)	(N/A N/A)			76.92 us	(x0.034 x0.023)	(N/A N/A)		
8	$g\bar{g} \rightarrow g\bar{g} + (n-2)g$	N/A					2.485 s	(x0.146)	(N/A)			3.424 s	(x0.705)	(N/A)		
							53.46 us	(x0.03 x0.021)	(N/A N/A)			1.29e+03 us	(x0.035 x0.025)	(N/A N/A)		
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A					N/A					2.778 s	(x0.191)	(N/A)		
												272 us	(x0.008 x0.006)	(N/A N/A)		
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A					N/A					N/A				
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A					N/A					N/A				
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A					N/A					N/A				
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A					N/A					N/A				
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A					N/A				
summary: gen		2.22	2.62	2.42	2.42		2.27	3.28	2.52	2.41		2.35	3.42	2.59	2.43	
		x0.076	x0.089	x0.083	x0.083	x0.082	x0.086	x0.146	x0.114	x0.114	x0.113	x0.115	x0.705	x0.234	x0.165	x0.255
summary: run		11.4	11.5	11.5	11.5		16.4	53.5	27.1	22.9		32.8	1.29e3	242	90.5	
		x0.014	x0.017	x0.016	x0.016	x0.016	x0.012	x0.04	x0.02	x0.015	x0.021	x0.008	x0.057	x0.022	x0.015	x0.03

Generic Full-Colour Performance Matrix (continued)

ID	base process	n=4				n=5				n=6			
1	$d\bar{d} \rightarrow Z + (n-1)g$	3.027 s	(x0.532)	(N/A)		12.99 s	(x1.50)	(N/A)		N/A			
		1000 us	(x0.02 x0.017)	(N/A N/A)		4.73e+04 us	(x0.013 x0.012)	(N/A N/A)					
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	3.086 s	(x0.363)	(N/A)		9.551 s	(x1.39)	(N/A)		N/A			
		658 us	(x0.019 x0.015)	(N/A N/A)		3.2e+04 us	(x0.011 x0.01)	(N/A N/A)					
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	2.515 s	(x0.155)	(N/A)		8.576 s	(x0.206)	(N/A)		N/A			
		162 us	(x0.01 x0.007)	(N/A N/A)		3.66e+03 us	(x0.005 x0.004)	(N/A N/A)					
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	2.361 s	(x0.125)	(N/A)		7.385 s	(x0.115)	(N/A)		N/A			
		106 us	(x0.008 x0.005)	(N/A N/A)		1.96e+03 us	(x0.004 x0.003)	(N/A N/A)					
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	3.42 s	(x0.269)	(N/A)		11.84 s	(x0.52)	(N/A)		N/A			
		818 us	(x0.009 x0.007)	(N/A N/A)		2.4e+04 us	(x0.004 x0.004)	(N/A N/A)					
6	$gg \rightarrow t\bar{t} + (n-2)g$	4.689 s	(x2.05)	(N/A)		86.75 s	(x4.18)	(N/A)		N/A			
		4.62e+03 us	(x0.078 x0.065)	(N/A N/A)		3.42e+05 us	(x0.081 x0.073)	(N/A N/A)					
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	4.172 s	(x0.56)	(N/A)		43.03 s	(x0.871)	(N/A)		N/A			
		963 us	(x0.049 x0.041)	(N/A N/A)		3.96e+04 us	(x0.045 x0.041)	(N/A N/A)					
8	$gg \rightarrow gg + (n-2)g$	14.12 s	(x3.76)	(N/A)		289 s	(N/A)	(N/A)		N/A			
		4.75e+04 us	(x0.04 x0.035)	(N/A N/A)		2.37e+06 us	(N/A N/A)	(N/A N/A)					
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	3.713 s	(x0.246)	(N/A)		17.01 s	(x0.242)	(N/A)		N/A			
		1.31e+03 us	(x0.005 x0.004)	(N/A N/A)		2.66e+04 us	(x0.002 x0.002)	(N/A N/A)					
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	4.373 s	(x0.096)	(N/A)		13.88 s	(x0.041)	(N/A)		N/A			
		106 us	(x0.016 x0.012)	(N/A N/A)		779 us	(x0.004 x0.004)	(N/A N/A)					
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	3.248 s	(x0.186)	(N/A)		8.084 s	(x0.3)	(N/A)		N/A			
		194 us	(x0.022 x0.017)	(N/A N/A)		2.1e+03 us	(x0.016 x0.014)	(N/A N/A)					
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	4.591 s	(x0.101)	(N/A)		10.99 s	(x0.048)	(N/A)		N/A			
		234 us	(x0.007 x0.005)	(N/A N/A)		1.3e+03 us	(x0.002 x0.002)	(N/A N/A)					
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	1.202 s	N/A		N/A	3.312 s	N/A		N/A			
		N/A	(8.21 5.16)	N/A		N/A	(67.5 55)	N/A					
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A				N/A				N/A			
summary: gen		2.36	14.1	4.44	3.57	7.38	289	43.3	12.4	N/A			
		x0.096	x3.76	x0.704	x0.258	x0.041	x4.18	x0.856	x0.3	N/A			
summary: run		106	4.75e4	4.81e3	738	779	2.37e6	2.41e5	2.53e4	N/A			
		x0.005	x0.078	x0.024	x0.018	x0.002	x0.081	x0.017	x0.005	x0.059			

Each cell compares raw generated-library AMPLICOL colour probes against PYAMPLICOL staged-DAG JIT O3 and C++ O3 at the same final-state multiplicity n . Cell slots are AMPLICOL, PYAMPLICOL JIT, and PYAMPLICOL C++ O3; each slot shows generation time above runtime per phase-space point. PYAMPLICOL runtime multipliers are (wall|core) relative to AMPLICOL. If no direct AMPLICOL reference exists, completed PYAMPLICOL slots show absolute timings. For NLC/full-colour tables, AMPLICOL reference values come from the dedicated raw generated-library colour driver `amplicol_color_library_probe`. Ratio colours are green below one, orange below two, and red otherwise. The summary rows give min|max|avg|med; multiplier summaries add sum, the ratio of paired summed PYAMPLICOL and AMPLICOL times. Missing and structural N/A cells are ignored. Validated rows use the same phase-space point and a 10^{-8} relative tolerance.

A red **VALIDATION FAILED** marker is shown next to any validated entry whose relative difference exceeds 10^{-8} .

Matrix run settings. Generated by `docs/result_matrix.py` with `-color-accuracy full`, from `result_matrix_full_data.json` into `result_matrix_full_table.tex`. AMPLICOL uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`, preserving NLC/full colour-order interferences. PYAMPLICOL uses Rusticol double precision, batch size 64, output chunk size 128, stage-local evaluator inputs, ten Horner iterations, Symbolica's default CPE iteration choice, and enlarged Horner/common-pair limits. JIT rows use SymJIT O3; timing runs use `-target-runtime 10`. Only C++ O3 generation is subject to the 15-minute compile cap.

Fortran colour-reference limitations. PYAMPLICOL generated these cells, but the current Fortran colour-reference path stops above two quark lines: $d\bar{d} \rightarrow u\bar{u} s\bar{s} + (n-4)g$, $n=4$; $d\bar{d} \rightarrow u\bar{u} s\bar{s} + (n-4)g$, $n=5$.

C++ O3 compile-budget gaps. These cells have completed AMPLICOL and PYAMPLICOL JIT rows but no completed C++ O3 row: $d\bar{d} \rightarrow Z + (n-1)g$, $n=1$: not run; $d\bar{d} \rightarrow Z + (n-1)g$, $n=2$: not run; $d\bar{d} \rightarrow Z + (n-1)g$, $n=3$: not run; $d\bar{d} \rightarrow Z + (n-1)g$, $n=4$: not run; $d\bar{d} \rightarrow Z + (n-1)g$, $n=5$: not run; $u\bar{d} \rightarrow W^+ + (n-1)g$, $n=1$: not run; $u\bar{d} \rightarrow W^+ + (n-1)g$, $n=2$: not run; $u\bar{d} \rightarrow W^+ + (n-1)g$, $n=3$: not run; plus 34 more localized entries.

Archived low-multiplicity validation fixture. The $n \leq 4$ LC/NLC/full-colour validation cells are mirrored in `tests/fixtures/result_matrix_references.json`. The integration test checks those cached same-point AMPLICOL/PYAMPLICOL values without a local Fortran run; `scripts/refresh_result_matrix_fixtures.py` refreshes the fixture from the matrix caches.

10 Why This Matches AmpliCol but Remains Python-Friendly

The Python layer owns graph construction, process metadata, deterministic phase-space steering, artifact generation, evaluator caching, and validation against AMPLICOL. It does not perform the hot current recursion component by component in production. The two production execution routes are now:

Python staged route: source fill $\rightarrow$ bounded stage evaluators $\rightarrow$ amplitude/raw-sum evaluator,

Rusticol route: one PyO3 call $\rightarrow$ Rust source fill and staged sweep $\rightarrow$ amplitude/raw-sum evaluator.

Thus PYAMPLICOL keeps AMPLICOL's essential optimization - shared currents across helicity parents and a topological current-table sweep - while replacing generated Fortran subroutines by saved Spensio/Symbolica evaluator artifacts. The `rusticol` path removes the remaining Python orchestration cost by loading the same process directory and executing the sweep in Rust.

The old fully monolithic compiled-DAG route is now deprecated and kept only as reference code. Its alias-based design was useful for exploring whole-graph compilation, but the production target is the staged DAG: Python generation plus a Rust eager-DAG runtime over saved Symbolica stage evaluators. This keeps the runtime schedule close to AMPLICOL's current-table library while leaving the graph compiler model-driven and process-generic.